

Java Programs with Loops

1. generate a right-angled number pattern using nested loops. Apply a decision statement to replace all prime numbers with the symbol * in the pattern.
 2. program to process a list of strings using loops and apply decision logic to separately count uppercase letters, lowercase letters, and digits.
 3. Vowel count and consonant count
 4. frequency of odd, even
 5. Multiplication table.
- give simple and easy separate program

Below are 5 simple and easy Java programs, each separate and beginner-friendly.
(Using loops + decision statements as asked)

1. Right-angled Number Pattern (Replace Prime Numbers with *)

Output example (for n = 5):

markdown

```
1
* *
* * *
* 4 * 6
* 4 * 6 *
```

Java Program

java

```
class PrimePattern {
    static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0) return false;
        }
        return true;
    }
}
```

```

    }

    public static void main(String[] args) {
        int num = 1;

        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                if (isPrime(num))
                    System.out.print("* ");
                else
                    System.out.print(num + " ");
                num++;
            }
            System.out.println();
        }
    }
}

```

2. Count Uppercase, Lowercase Letters & Digits in a String

Java Program

```

java

class CountCharacters {
    public static void main(String[] args) {
        String s = "Hello123WORLD";

        int upper = 0, lower = 0, digit = 0;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (ch >= 'A' && ch <= 'Z')
                upper++;
            else if (ch >= 'a' && ch <= 'z')
                lower++;
            else if (ch >= '0' && ch <= '9')
                digit++;
        }
    }
}

```

```
        System.out.println("Uppercase letters: " + upper);
        System.out.println("Lowercase letters: " + lower);
        System.out.println("Digits: " + digit);
    }
}
```

3. Vowel Count and Consonant Count

Java Program

```
java

class VowelConsonantCount {
    public static void main(String[] args) {
        String s = "programming";
        int vowels = 0, consonants = 0;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);

            if (ch >= 'a' && ch <= 'z') {
                if (ch == 'a' || ch == 'e' || ch == 'i' ||
                    ch == 'o' || ch == 'u')
                    vowels++;
                else
                    consonants++;
            }
        }

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
    }
}
```

4. Frequency of Odd and Even Numbers

Java Program

```
java
```

```

class OddEvenCount {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};
        int even = 0, odd = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0)
                even++;
            else
                odd++;
        }

        System.out.println("Even count: " + even);
        System.out.println("Odd count: " + odd);
    }
}

```

5. Multiplication Table

Java Program

```

java

class MultiplicationTable {
    public static void main(String[] args) {
        int n = 5;

        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}

```

If you want:

- Same programs in C / Python
- Very short exam-ready versions
- With user input (Scanner)

1. Compare free and open source software by applying their principle.
 2. Foss licensing model and demonstrate copy left principle
 3. Steps in setting up a dual boot with linux and other os.
 4. Linux process management using system calls such as fork(), exec(), and wait ().
 5. Perl program demonstrating the use of different variable types n syntax.
 6. Compare gpl, agpl. Lgpl and analyse their application
 7. Process management in Linux with examples.
 8. X window system architecture to show how to configure it step by step in Linux
 9. List, hashes with examples.
- give answers for all the above 9 questions answer in 10 marks

Below are **well-structured 10-mark answers** for **all 9 questions**, written in **simple, exam-oriented language** with headings and examples.

1. Compare Free Software and Open Source Software using their principles (10 Marks)

Free Software (FSF – Richard Stallman)

Free software focuses on **user freedom**, not price.

Four Principles (Freedoms):

1. Freedom to run the program for any purpose
2. Freedom to study and modify the source code
3. Freedom to redistribute copies
4. Freedom to distribute modified versions

👉 Ethical and philosophical focus on **user rights**.

Open Source Software (OSI)

Open source focuses on **collaborative development and quality**.

Principles:

- Source code must be available
- Free redistribution
- Derived works allowed
- No discrimination against users or fields

👉 Practical and business-friendly approach.

Comparison Table

Aspect	Free Software	Open Source
Focus	User freedom	Development efficiency
Philosophy	Ethical	Practical
Founder	Richard Stallman	Eric Raymond
License Example	GPL	Apache, MIT
Commercial use	Allowed	Allowed

Conclusion

Both promote source code access, but **Free Software stresses freedom**, while **Open Source stresses collaboration and quality**.

2. FOSS Licensing Model and Copyleft Principle (10 Marks)

FOSS Licensing Model

FOSS licenses define how software can be:

- Used

- Modified
- Distributed

Examples:

- GPL
 - LGPL
 - MIT
 - Apache
-

Copyleft Principle

Copyleft ensures:

- Modified versions **must remain open source**
- Same license must be applied to derived works

👉 Prevents proprietary misuse.

Example: GNU GPL

- If you modify GPL software
 - You **must release source code**
 - You **must use GPL again**
-

Benefits

- Protects user freedom
 - Encourages open collaboration
 - Prevents closed-source exploitation
-

Conclusion

Copyleft is the backbone of FOSS, ensuring software remains **free forever**.

3. Steps to Set Up Dual Boot with Linux and Another OS (10 Marks)

Step-by-Step Process

- 1. Backup Data**
 - Prevents data loss
 - 2. Check System Requirements**
 - CPU, RAM, disk space
 - 3. Create Free Disk Space**
 - Shrink existing partition
 - 4. Create Bootable USB**
 - Use tools like Rufus / Balena Etcher
 - 5. Disable Secure Boot (if needed)**
 - From BIOS/UEFI
 - 6. Boot from USB**
 - Select "Install Linux"
 - 7. Choose Installation Type**
 - "Install alongside Windows"
 - 8. Partition Disk**
 - Root (/), Swap, Home (/home)
 - 9. Install GRUB Bootloader**
 - Allows OS selection at startup
 - 10. Restart System**
 - Choose OS from GRUB menu
-

Conclusion

Dual boot allows multiple OS usage on one system efficiently.

4. Linux Process Management using fork(), exec(), wait() (10 Marks)

Process

A process is a program under execution.

System Calls

fork()

- Creates a new child process
- Returns:
 - 0 → child
 - PID → parent

exec()

- Replaces current process with new program
- Does not create new process

wait()

- Parent waits for child to finish
 - Prevents zombie processes
-

Example Flow

1. Parent calls `fork()`
 2. Child executes new program using `exec()`
 3. Parent waits using `wait()`
-

Conclusion

These calls help manage multitasking and synchronization in Linux.

5. Perl Program Demonstrating Variable Types and Syntax (10 Marks)

Perl Variable Types

1. **Scalar (\$)** – Single value
 2. **Array (@)** – List of values
 3. **Hash (%)** – Key-value pairs
-

Perl Program

```
perl
```

```
#!/usr/bin/perl
use strict;
use warnings;

my $name = "Linux";      # Scalar
my @nums = (1, 2, 3);    # Array
my %info = ("OS" => "Ubuntu", "Version" => "22.04"); # Hash

print "Name: $name\n";
print "Array elements: @nums\n";
print "OS: $info{OS}\n";
```

Conclusion

Perl provides flexible data handling using simple symbols.

6. Compare GPL, AGPL, LGPL and Analyze Their Applications (10 Marks)

License	Description	Use Case
GPL	Strong copyleft	Open-source apps
LGPL	Weak copyleft	Libraries
AGPL	Network copyleft	Web apps
MIT	Permissive	Commercial software

Key Differences

- GPL → All derivatives must be open
 - LGPL → Allows proprietary linking
 - AGPL → Source must be shared even over network use
-

Conclusion

License choice depends on **software usage and distribution model**.

7. Process Management in Linux with Examples (10 Marks)

Key Concepts

- Process ID (PID)
 - Parent & child process
 - States: Running, Sleeping, Zombie
-

Commands

- `ps` → List processes
 - `top` → Real-time monitoring
 - `kill` → Terminate process
 - `nice` → Priority control
-

Example

```
bash
```

```
ps -ef
```

```
kill 1234
```

Conclusion

Linux provides powerful tools for efficient process control.

8. X Window System Architecture and Configuration Steps (10 Marks)

X Window Architecture

1. **X Server** – Controls display & input
2. **X Client** – Applications
3. **Window Manager** – GUI control

4. Display Hardware

Configuration Steps

1. Install X server

```
bash  
  
sudo apt install xorg
```

2. Install window manager

```
bash  
  
sudo apt install gnome
```

3. Configure display

```
bash  
  
xrandr
```

4. Start X session

```
bash  
  
startx
```

Conclusion

X Window separates GUI logic from applications, enabling flexibility.

9. Lists and Hashes with Examples (10 Marks)

List

- Ordered collection of values

Example (Perl):

```
perl
```

```
my @colors = ("red", "green", "blue");  
print $colors[0];
```

Hash

- Key-value pair collection

Example (Perl):

```
perl  
  
my %student = ("name" => "Ram", "age" => 21);  
print $student{name};
```

Comparison

Feature	List	Hash
Access	Index	Key
Order	Yes	No

Conclusion

Lists and hashes help store and retrieve structured data efficiently.

If you want:

- Shorter exam answers
- Diagrams for X Window
- Handwritten-style notes
- PDF format

Just tell me 👍

Difference between GPL ,AGPL and LGPL with examples give answer for 10 markd

Below is a **clear, exam-ready 10-mark answer** on the **Difference between GPL, AGPL and LGPL**, written in **simple language with examples**.

Difference between GPL, AGPL and LGPL (10 Marks)

Introduction

GPL, AGPL, and LGPL are **Free and Open Source Software (FOSS) licenses** created by the **Free Software Foundation (FSF)**.

They are based on the **copyleft principle**, which ensures that software and its modified versions remain free and open source.

1. GNU General Public License (GPL)

Definition

GPL is a **strong copyleft license**. Any software derived from GPL-licensed code **must also be released under GPL**.

Key Features

- Source code must be made available
- Modified versions must be open source
- Same GPL license must be used for redistribution
- Not suitable for proprietary software

Example

- **Linux Kernel**
- **GCC Compiler**

👉 If a company modifies the Linux kernel, it must release the modified source code under GPL.

2. GNU Affero General Public License (AGPL)

Definition

AGPL is similar to GPL but includes **network use protection**.

It ensures source code is shared even when software is used **over a network**.

Key Features

- Strong copyleft + network clause
- Source code must be provided to users accessing software via the internet
- Prevents SaaS companies from hiding modifications

Example

- MongoDB (earlier versions)
- Nextcloud

👉 If a web application runs AGPL software, users accessing it online can request the source code.

3. GNU Lesser General Public License (LGPL)

Definition

LGPL is a **weak copyleft license**, mainly used for **libraries**.

Key Features

- Allows linking with proprietary software
- Modifications to the library must be open source
- Application using the library can remain closed source

Example

- glibc
- FFmpeg libraries

👉 A proprietary app can use an LGPL library without releasing its own source code.

Comparison Table

Feature	GPL	AGPL	LGPL
Copyleft strength	Strong	Very Strong	Weak
Network use clause	✗ No	✓ Yes	✗ No
Suitable for libraries	✗ No	✗ No	✓ Yes
Proprietary linking	✗ Not allowed	✗ Not allowed	✓ Allowed

Feature	GPL	AGPL	LGPL
Source sharing required	On distribution	On network use	Only for library changes

Applications / Use Cases

- **GPL:** Desktop applications, OS kernels
 - **AGPL:** Web services, cloud platforms
 - **LGPL:** Shared libraries and frameworks
-

Conclusion

- **GPL** ensures complete freedom of software
- **AGPL** extends freedom to network-based software
- **LGPL** balances open source with commercial usage

👉 Choosing the right license depends on **how the software is used and distributed**.

If you want this:

- **Short 5-mark version**
- **One-page notes**
- **Diagram-based explanation**

Just tell me 😊